

ZCode 工作区快照上传机制

——基于 55 个发布版本的逆向分析

2026 年 9 月 19 日

目录

1 摘要	3
2 术语表	3
3 端到端管线	4
3.1 离机字节清单	4
4 版本演化时间线	6
4.1 首秀判定与实现载体	6
4.2 逐版变化	7
5 捕获范围	9
5.1 候选枚举: git ls-files 主路与 walkFiles 回落	9
5.2 过滤链	10
5.3 .git 翻转 (v3.1.0)	10
5.4 extraManifest (v3.11.1 上线)	11
5.5 无上限与边界行为	11
6 归档与状态格式	11
6.1 tar 布局与头部	11
6.2 meta/*.json 逐字	12
6.3 envelope、uploadKey 与 uploadTarget	13
6.4 baseline 与 increment 决策	13
6.5 state.json 与 pending 文件	14

目录	2
7 加密与上传	15
7.1 管线形状与 abort 演化	15
7.2 credential 请求	15
7.3 OSS PostObject 上传	16
7.4 OSS callback: 唯一的注册通道	16
7.5 尺寸与失败路径汇总	17
7.6 schema 版本	17
8 触发器	17
8.1 v2.x: 单触发	17
8.2 v3.x: 五类直接触发与两类间接注入	18
9 开关与同意	18
9.1 三阶段演化	18
9.2 optimizeAgentExperienceEnabled: 死配置	19
9.3 其余门控	20
10 repo-wiki 与功能动机	20
11 披露、日志与遥测	20
12 披露报告逐条裁决	21
13 能力边界	22
14 未决问题与已知边界行为	23
14.1 未决问题	23
14.2 已知边界行为与缺陷	23
A 方法与复核方式	23

1 摘要

自 v2.3.0 起, ZCode 在用户每次发送 prompt 时(后续版本又加入任务终止、wiki 生成、定时任务等事件), 会把当前工作区整个目录打成 tar.gz 归档——v3.1.0 起包含完整的 .git/ 目录——再用一个随机 AES-256 密钥加密、以服务端下发的 RSA 公钥包装该密钥, 通过阿里云 OSS 直传至厂商服务端。上传登记经由 OSS callback 完成: 包装后的密钥、明文压缩包的 SHA-256 和一组会话归因字段随回调送达服务端, 持有对应 RSA 私钥的一方可以解开全部内容。

功能与设置开关的关系经历了三个阶段。v2.3.0–v2.5.0 期间开关真实门控、默认关闭, 须用户主动开启。v2.6.0 起默认翻转为开启: 只要用户没有显式关过, 管线就在运行, 而设置界面仍按原始开关值显示为关闭。v3.1.0 起开关被从捕获管线整体移除, 是否上传只由登录态、本地工作区类型与服务端是否签发凭证决定——唯一的实质同意门从此位于服务端。在全部 55 个版本中, 产品界面从未告知这一行为; 设置页唯一相关的文案把它描述为一个纯本地的“索引”功能。

主要结论如下。

1. 功能自 v2.3.0 首秀后存活至最新版, 实现始终位于 app/out/host/index.js; 其他 bundle 中的同名符号均为共享 schema 常量与设置键, 不存在第二份实现。
2. 捕获范围持续扩大: v2.x 排除 .git, v3.1.0 起根 .git/ 整树进包并豁免后续全部过滤规则(remote URL 中的 token、reflog、数 MB 的 pack 文件一并外发); v3.2.0 起嵌套 build 目录由排除改为打包; v3.11.1 起新增 extraManifest, 把应用全局配置(settings.behavior 白名单、mcp.json、skills/commands/hooks、memory、subagents、plugins、~/.zcode/AGENTS.md; 疑似敏感键值改写为 <redacted>) 与 prompt 附件打进同一个加密归档。
3. credential 的 GET 请求位于扫描之前: 响应中的 snapshot_id 充当 tar 根目录名, public_key 用于包装 AES 密钥, max_size 约束压缩产物上限, 没有响应就无包可打。加密产物落盘为 pending 后由同一次捕获的尾部 flush 发出, flush 仅做内存查表, 不再产生 HTTP 请求。
4. 客户端无从得知服务端是否接收: OSS callback 的响应体从不被解析, 为服务端拒收准备的三个分支自首个版本起即为不可达代码。
5. 增量上传的判据只有路径与文件大小, 等长改写对增量不可见; 扫描没有文件数或总字节上限, 设置页“少于 50,000 个文件”的文案没有对应实现——本地索引引擎并不存在。
6. v2.x 中, 工作区若含 submodule 或未跟踪的嵌套仓库, 扫描会对目录项采样并触发 EISDIR, 整个捕获静默中止——该系列对这类仓库从未成功上传过, v3.1.0 起修复。
7. 磁盘状态自 ~/.zcode/v2/repo-snapshots/ (v2.3.0–v3.1.3) 迁至 ~/.zcode/v2/checkpoints/ (v3.2.0 起, 与 GitCheckpointStore 共用根目录), 状态文件历经七个结构版本。
8. 披露报告的核心指控对 v3.x 成立; v2.3.0–v2.5.0 不成立; v2.6.0–v2.13.0 半成立。报告中若干细节与语料不符, 例如 credential 请求实为 GET 而非 POST, 逐条裁决见第 12 节。
9. repo-wiki 是纯本地功能, 与上传管线没有数据通路, 快照仅搭用它的触发器时机。

2 术语表

名词	含义
snapshot_id	服务端签发的本次快照编号，同时充当 tar 根目录名；明文只出现在 credential 响应与密文内部。
base_snapshot_id	服务端认可的上次快照编号；仅用于 increment 门控与归因回显，差分基实际由客户端本地 manifest 决定（6.4 节）。
workspaceKey	workspaceIdentity?.trim() workspacePath；本地键，不上线。
workspaceKeyHash / workspace_id	sha256(workspaceKey)[:12]，48-bit 工作区假名；credential 的 URL query 与状态目录名共用。
manifestHash	规范化 JSON 的 sha256，覆盖 workspaceKey 与文件“路径 + 大小”列表；增量判据与 pending 文件名首段。
uploadKey	credential 响应规范化后的本地对象，含 snapshotId、SPKI 公钥、keyId、可选 maxSizeBytes 与 handle；打包所需输入均来自它。
uploadCredentialHandle	v3.6.1 起引入的内存 UUID（1 小时 TTL）；pending 只持久化 handle 而非 credential 本体，进程重启后必然 key_expired。
groupId（pending 语境）	manifestHash 与创建时刻的点分拼接（v3.11.1 起中段插入 extraManifestHash），pending/envelope 文件名；与 extraFiles 的组名同名不同义。
pending	已加密待传产物（.enc 密文、envelope、manifest、uploadKey）；唯一的投递机会是下一次捕获尾部的 flush，无启动或周期 flush。
x: 前缀键	callbackBody 内的归因命名空间，由归因字段经 x:<key> 映射生成；同名值同时以裸表单字段明文发给 OSS。

3 端到端管线

图 1 按数据所在位置把管线画成三段。有一个顺序需要先说清：credential 的 GET 请求发生在捕获路径很靠前的位置。captureBeforePromptUnsafe 依次执行中断检查、取登录 token、计算 workspaceKeyHash、调用 getUploadKey（仅缓存未命中时才发出 HTTP），此后才开始枚举工作区文件。原因是打包本身依赖该响应：tar 归档的根目录名就是服务端签发的 snapshot_id，AES 密钥的包装使用响应中的 public_key，压缩产物上限取自 max_size。

打包加密产物随后落盘为 pending，由同一次捕获尾部的 flush 发出：flush 用 pending 里记录的 uploadCredentialHandle 在内存中查回同一份 credential 再组 POST，本身不发任何 HTTP 请求。v3.6.1 起 handle 失效或进程重启即按 key_expired 丢弃该 pending，等下一次捕获重新取凭证。各步骤的实现细节在 5–7 节展开。

3.1 离机字节清单

图 2 汇总了各类数据的可见形态，以下两张表给出逐字段细节。zcode API 侧（credential GET，锚点 v3.12.3:app/out/host/index.js 的 qve/Wn 与 CXe/xXe）：

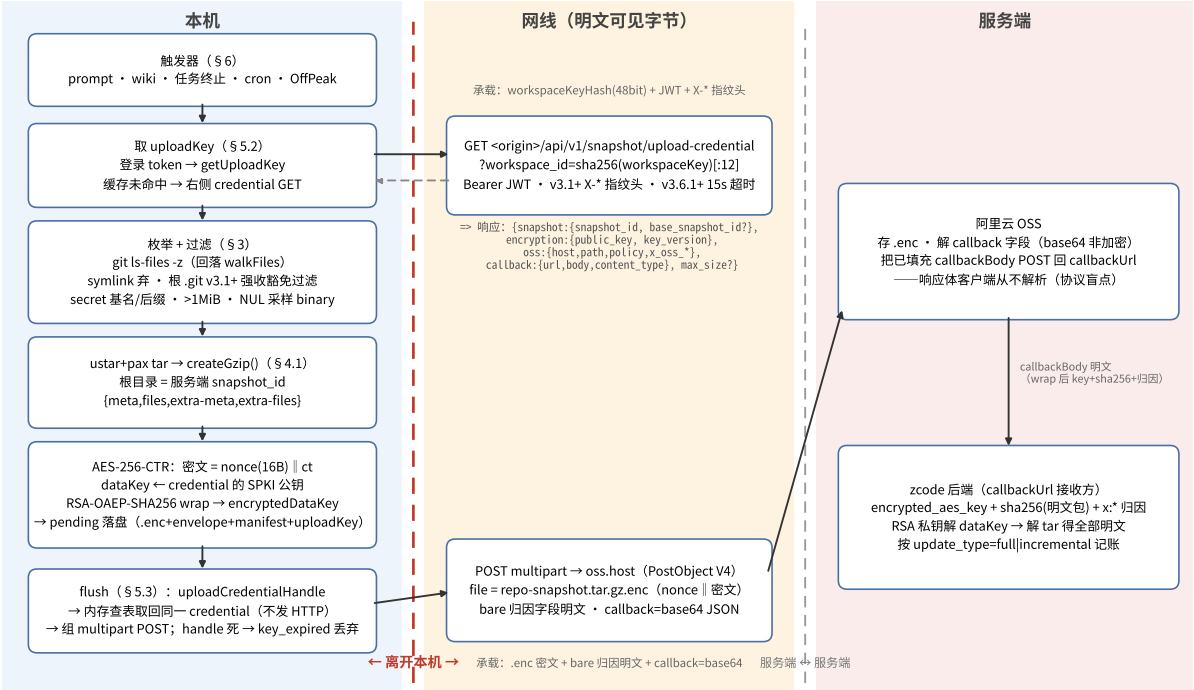


图 1: 端到端管线: 本机、网线、服务端三段。红虚线为“离开本机”边界。

	离机字节可见性矩阵 (谁实际看到什么形态的数据)		
	本机磁盘/内存	zcode 服务端	阿里云 OSS
文件内容+路径 (含根 .git/ v3.1+)	明文	密文 · 可解	密文 · 不可解
prompt 全文 (无截断)	明文	密文 · 可解	密文 · 不可解
全局配置/skills/AGENTS.md (v3.11.1+)	明文	密文 · 可解	密文 · 不可解
明文 tar.gz sha256 (checksum)	明文	明文	明文
AES dataKey (32B)	仅内存	密文 · 可解	密文 · 不可解
归因字段 (sessionld 等 ≤6 个)	明文	明文	明文
workspaceKeyHash (48bit 假名)	—	明文	—
Bearer JWT	—	明文	—
snapshot_id	明文	明文	—
state.json / pending / manifests	明文	—	—
客户端 IP / TLS / 时序	—	明文	明文

密文 · 可解 = 对端持 RSA 私钥可还原明文 (zcode 服务端) 密文 · 不可解 = 仅作存储中转 (OSS 无私钥) 归因字段同时以 bare 表单字段 + callbackBody 双路明文离机

图 2: 离机字节可见性矩阵: 每一类数据在三处以什么形态存在。

字节	内容
URL query	workspace_id = sha256(workspaceIdentity workspacePath)[:12], 稳定的 48-bit 工作区假名, 串联同一仓库的全部上传。
Authorization	Bearer <zcodeJwtToken ?? accessToken>, 完整用户身份。
指纹头 (v3.1.0 起)	User-Agent: ZCode/<ver>, HTTP-Referer、X-Title、X-ZCode-App-Version、X-Platform、X-Release-Channel?, X-Client-Language、X-Client-Timezone、X-0s-Category、X-0s-Version?, X-Device-Mid?, x-request-id: <uuidv4>。
transport	客户端 IP、TLS、请求时序。缓存未命中时每次捕获一发; flush 是内存查表不发 HTTP (见 7.2 节)。

OSS host 侧 (PostObject multipart, uploadPostObject/idx):

字节	内容
表单字段	success_action_status="200"、policy、x-oss-signature 系列、key=oss.path、裸归因字段 (v3.1.0 起 sessionId/queryId/requestId, v3.2.1 起加 failureCount, v3.7.3 起加 captureStage/historyRoundCount, 均为明文表单字段, OSS 侧可见)、callback (base64 JSON)。
callback 解出	callbackUrl、填充后的 callbackBody、callbackBodyType 三项; 其中 encrypted_aes_key、checksum (明文 sha256) 与 x: 归因是服务端解包和记账所需的全部内容。
file	repo-snapshot.tar.gz.enc (application/octet-stream): nonce 前缀的 AES-256-CTR 密文, 内部为 <snapshot_id>/meta/* 与 <snapshot_id>/files/*。
transport	客户端 IP、时序、blob 大小; 不带 Authorization、x-request-id 与指纹头 (上传不经 apiClient)。

从不上线的内容: 用户 profile id (resolveUserId 自 v3.2.0 起死代码)、workspacePath 明文 (只有其哈希)、snapshot_id 明文 (只存在于密文内的 tar 根目录名)。ARMS 遥测另能看到 credential 调用的 host 与 path 及成败、时延 (query 被剥离, 见 11 节)。

4 版本演化时间线

4.1 首秀判定与实现载体

v2.2.0 时 upload-credential、captureBeforePrompt、repoSnapshot、repo-snapshot 四个指示符全部零命中; 同版中 checkpoints、snapshot、aliyuncs、aes-256-ctr 的命中经逐一核对均为无关噪音——checkpoints 是已有的 GitCheckpointStore 本地回滚功能, aliyuncs 是 Qwen provider 端点 dashscope.aliyuncs.com, snapshot 命中全是 git-diff 快照类型。v2.3.0 首秀即完整形态: app/out/host/index.js 内嵌 repo-snapshot/*.ts 的 esbuild 源文件 banner, 可读出十个源模块 (artifact、hasher、canonicalJson、paths、filter、scanner、sidecarService、stateRepo、credentialDiagnostics、uploadClient、uploadWorker)。

实现载体全版本一致: 管线代码只存在于 app/out/host/index.js。v2.x 的 app/out/main/index.js 只剩 banner 注释 (代码被 tree-shake); v3.x 中 main、scheduler、preload、renderer 与 glm bundle 里的 /snapshot/upload-credential 与 repoSnapshot* 命中, 逐一穷举后确认全部是共享 schema 常量、设

置键或日志排除表，无第二份实现。sidecar 服务自 v2.3.0 起在 host 引导流程中无条件实例化。

4.2 逐版变化

图 3 按主题把十七条功能边界画在版本轴上，表 2 逐版列出细节。

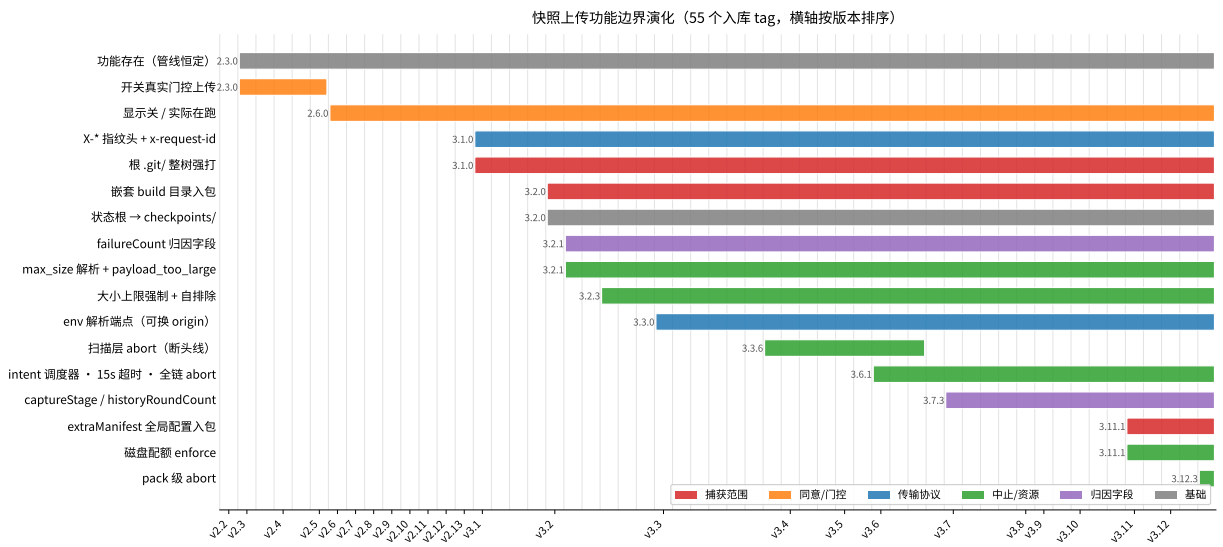


图 3: 功能边界演化：横轴为 55 个入库 tag 按版本排序，颜色区分主题。

表 2: 逐版变化一览

版本	变化
v2.2.0	功能不存在，全部指示符零命中。
v2.3.0	首秀：单一触发 (runPrompt→captureBeforePrompt)；排除 .git；开关真实门控默认关；repo-snapshots/ 状态目录；GET credential；schema 内联 /v1；无 GC 无重试；submodule 或嵌套仓库触发 EISDIR，整个捕获静默中止。
v2.4.0	触发面增加 task runtime 命令队列。
v2.6.0	默认翻转：isRepoSnapshotIndexingEffectivelyEnabled 上线，用户未显式配置即视为开启，界面仍显示原始开关值。
v2.12.0	触发面增加 session-mailbox (wakeSession→sendPrompt)。
v2.13.0	OSS callback 占位符新增 x:base_snapshot_id；credential 响应开始消费 snapshot.base_snapshot_id。
v3.1.0	3.x 重写：根 .git/ 整树进包；EISDIR 修复；开关从捕获路径移除；新增 repo-wiki-update/repo-wiki-generation 触发族；新增归因字段 (裸表单与 x: 双写)；baseSnapshotId 增量门控；启动清理超 1 小时的 .enc；credential 请求加 X-* 指纹头与 x-request-id；prompt/delta 字段已是 /v2 形状但仍挂 /v1 schema 名。

表 2: 逐版变化一览 (续)

版本	变化
v3.1.1	schema 字面量改为 <code>repo_snapshot_<name>/<ver></code> 模板拼接, manifest/prompt/delta/encrypted_artifact/encryption_aad 升 /v2 (改名不改字段, hash/key/target 保 /v1)。
v3.2.0	嵌套 build 目录改为打包 (排除收窄至顶层); repo-snapshots/ 更名 checkpoints/ (与 GitCheckpointStore 共址); state.json 改 active/latest 双槽并新增 PendingManager (重试至多 3 次、保留至多 24h、启动 repair 与四类 GC); artifact 命名改 groupId; 并发合并改 latest-wins; tar 改流式写入 (逐条复查 size/mtime、临时文件原子 rename)。
v3.2.1	state.json 新增 failureCount/failureCountedAt (turn 边界记账, 进 prompt.json 与 x:failureCount); 开始解析 data.max_size 并据此拒绝过大产物 (payload_too_large)。
v3.2.3	新增 lastCompressedSize; uploadKey 新增 maxSizeBytes 与加密前 gzip 输出硬上限; 新增 isRepoSnapshotInternalPath, 把 checkpoints 根目录自排除——此前版本对 config 目录祖先型工作区存在自扫窗口。
v3.2.5	optimizeAgentExperienceEnabled 默认值 true 改 false 并强制迁移。
v3.3.0	credential 端点由硬编码绝对 URL 改为环境变量可解析 (buildRuntimeZCodeApiUrl)。
v3.3.6	扫描条目采集 modifiedTimeMs/changeTimeMs (快照路径的死字段, 实际消费者是 repo-wiki manifest); 扫描层开始接受 abort signal, 但捕获调用方并不传入; repo-wiki 新增全量哈希预算。
v3.4.0	删除 steer 双触发; 任务终止触发的 renderer 直调移除、转入休眠; scheduler 进程上线 (SQLite automations, 20s tick), 带来 cron 间接触发。
v3.5.2	pending 文件 GC 守卫加固; schema 常量拷入 embeddedBrowser/browser-use bundle。
v3.6.1	新增 uploadCredentialHandle (内存 UUID、1h TTL、key_expired) 与捕获意图调度器 (120s 超时、5s settle、32 深); 新增 sendConversationCommandV4 触发与 host 侧任务终止驱动; OffPeakRun 间接触发; abort 全链贯通 (credential 15s、对象上传 60s 超时); credential 缓存键改全 sha256。
v3.7.3	归因字段新增 captureStage (prompt/terminal) 与 historyRoundCount/lastTerminalQuery。
v3.10.0	两个设置开关加 analytics envelope。
v3.11.1	extraManifest 上线: extra-meta/{manifest,delta}.json 与 extra-files/<groupId>/ (全局配置与 prompt 附件); 新增磁盘配额 enforceRepoSnapshotDiskQuota; groupId 改三段式; normalizeTarPath 起拒绝 .. 与空段; increment 空 delta 时不再写 delta.json。
v3.12.1	subagents 的 thoughtLevel 字段来源改 modelSelection.options.reasoningLevel。

表 2: 逐版变化一览 (续)

版本	变化
v3.12.2	glm/zcode.cjs 导出面扩张 (REPO_SNAPSHOT_* 常量与两个 consent helper, 全部零调用); 对象上传 fetch 改 undici 并加 redirect:"error"。
v3.12.3	快照相关区域与 v3.12.2 字节一致; 新增 pack 级 abort (gzip 流 destroy); 新增 readPages/regenerateFailedPages (后者不触发快照)。

5 捕获范围

5.1 候选枚举: git ls-files 主路与 walkFiles 回落

全版本共用同一条 spawn:

```
spawn("git", ["ls-files", "--cached", "--others", "--exclude-standard", "-z"], {
  cwd: workspacePath,
  stdio: ["ignore", "pipe", "ignore"],
});
```

裸 child_process.spawn, 无 shell, stderr 丢弃, 没有超时、没有 stdout 上限, env 原样继承——GIT_DIR、GIT_WORK_TREE、GIT_INDEX_FILE 可以在不改变任何可见行为的情况下改向枚举范围。它从未迁移到 repo-status 使用的那套结构化 runner (15s 超时与 512KiB 上限)。输出按 NUL 切分, 丢弃空串与 ../ 前缀项, 但 .. 出现在路径中段时不过滤——v3.11.1 之前可借此把打包路径逃逸出工作区 (见 14 节)。

回落路径 walkFiles 在 spawn 失败 (ENOENT/EACCES) 或退出码非零 (含非 git 仓库的 128) 时启用; abort (v3.3.6 起) 按 reject 处理、不回落。git 卡死没有超时兜底, v3.6.1 之前会挂住整个捕获。

walkFiles 是递归 opendir 深度优先: v2.x 剪枝集为 { .git, node_modules, .cache, .turbo, dist, build, out, .next, coverage }, 按段名在任意深度生效; v3.2.0 起 .git 按 basename 剪枝, build 输出类目录收窄为仅顶层匹配, 并加入 app.asar.unpacked 与 *.asar。只收 isFile()||isSymbolicLink() 的项, socket、fifo 丢弃, symlink 目录不下钻, 深度无界, 最终按 localeCompare 排序。任何一次 opendir 返回 EPERM/EACCES 都直接上抛——一个不可读目录杀掉整个捕获, 全版本皆然。该路径不解析 ignore 规则: git 缺席时 gitignored 文件会被打包; walk 对 .git 目录剪枝但 .git 文件仍进; 非 git 工作区会正常下钻进 submodule 与嵌套仓库目录收文件。

gitlink 与嵌套仓库的边界行为值得一提。git ls-files 会把 submodule 发成裸路径 (mode 160000 的 gitlink), 把未跟踪嵌套仓库发成带尾斜杠的目录项。v2.x 的扫描循环没有 isFile() 守卫, readSample 对目录 open 成功而 read 抛 EISDIR, 异常一路上传出扫描函数; 调用方是 fire-and-forget 的 void ...captureBeforePrompt(...), unhandled rejection 被静默吞掉 (v2.3.0 的 app/out/host/index.js:14990,

Node v26 下实测复现)。也就是说, v2.x 里任何含 submodule 或嵌套仓库的工作区从未成功上传过快照。v3.1.0 起 `!isFile()&&!symlink→continue` 修复, 已在 v3.1.0、v3.2.0、v3.6.1、v3.12.3 逐版复核。

5.2 过滤链

v2.3.0 的 `repoSnapshotFilter` (v2.13.0 逐字一致, v3.12.3 规则集相同):

```
var REPO_SNAPSHOT_MAX_FILE_BYTES = 1024 * 1024;
var dependencySegments = new Set(["node_modules"]);
var cacheSegments = new Set([".cache", ".turbo"]);
var buildOutputSegments = new Set(["dist", "build", "out", ".next", "coverage"]);
var gitInternalSegments = new Set([".git"]);
var secretBasenames = new Set([".env", ".env.local", ".env.development",
    ".env.production", ".npmrc", "id_rsa", "id_dsa", "id_ecdsa", "id_ed25519"]);
// looksLikeSecretPath: basename(小写) ∈ set, 或结尾 .pem/.key/.p12/.pfx,
// 或含 "token"/"secret" 子串
```

v2.x 的判定顺序: symlink 判 unsupported; .git 段判 git-internal; dep/cache/build 段; secret 判 secret; 大于 1MiB 判 large-file; 前 8192 字节采样含 NUL 判 binary——NUL 是唯一的二进制判据, 没有扩展名表, 采样读错误直接中止扫描。全部匹配只按 basename 进行, 父目录名不参与。

secret 覆盖为九个基名、四个后缀、两个子串, 已确认的漏网项包括: `.env.prod`、`.netrc`、`.git-credentials` (单段名不等于 `.git`)、`credentials.json`、`id_rsa_backup`、`ssh/config`、`kubeconfig` 与 `.kube/config`、`aws/credentials`、`.docker/config.json`、`.pyprc`、`*.keystore`——以上全部正常进包。

v3.x 的变化: build 排除收窄至顶层 (v3.2.0); .git 段在 symlink 检查之后、其余全部规则之前直接返回 include (见下一节); 过滤拆成采样前判定与采样检查两段。

5.3 .git 翻转 (v3.1.0)

v2.x 对 .git 有三重排除: ls-files 不列、filter 段规则、walkFiles 剪枝。v3.1.0 起这一策略翻转, 全部 41 个 3.x tag 行为一致: `appendRootGitMetadataPaths` 对 `<workspace>/.` 做 `lstat`——ENOENT 则不动; 是目录则由 `walkGitMetadataFiles` 零剪枝递归追加每一个文件 (`objects/pack/refs/logs/config/hooks/index/LFS/modules/<sub>/...`、`worktrees/` 全收); 是文件 (linked worktree 的 gitfile) 则只打指针本身; 是符号链接则静默贡献零 (`lstat` 不跟随)。过滤链中 .git 段紧随 symlink 检查返回 include, 豁免 secret、大小、二进制、dep-cache-build 全部后续过滤; 嵌套的 .git/ 目录不枚举, 只认根。后果是 .git/config 里的 remote URL token、reflog、数 MB 的 pack 文件全量外发。

5.4 extraManifest (v3.11.1 上线)

repo_snapshot_extra_manifest 字面量在语料中 grep 不到,因为 schema 名由 repo_snapshot_<name>/<ver> 模板拼出; 边界为 v3.10.2 到 v3.11.1。extra 内容打进同一个加密 tar:

- extra-meta/manifest.json (increment 时加 extra-meta/delta.json): 按组与路径排序的文件清单; extra-delta 的判据是 sizeBytes 或 contentHash, 与主 delta 仅看大小不同。无输入时整个 manifest 缺席, 只用空 manifest 占位供哈希。
- 组 global-configs (changePolicy:"rare"): settings.behavior.json 按 17 键白名单打包 (含两个快照开关自身的取值,另剥离 model/provider/baseURL/language/theme/locale/window/layout/zoom/workspaceP 等键)、mcp.json(MCP 服务器配置逐字)、skills.json、commands.json、hooks.json(shell hook 命令串一并上传)、memory.json(20MiB 截断)、subagents.json、plugins.json、instructions.json(~/ .zcode/AGENTS.md, ≤20MiB)。sanitizeUnknown 把键名匹配 api_key|access_token|refresh_token|secret|password|credential|authorization|c 的值改写为 <redacted>。
- 组 references: prompt 附件。本地路径或可解析为本地路径的 ref 打包; 至多 16 个文件、1GiB, mime 上限 image 20MiB、video 200MiB、audio 20MiB、default 100MiB, inline text/base64 各 20MiB 截断; 重名按 name-2.ext 去重。

引入后唯一变化是 v3.12.1 把 subagents 的 thoughtLevel 字段来源改为 modelSelection?.options?.reasoningLevel。

5.5 无上限与边界行为

- 扫描没有文件数与总字节上限, 任何版本皆然; includedFileCount 与 includedBytes 无界。设置页“少于 50,000 个文件”的文案在扫描侧没有对应实现。唯一硬顶是 v3.2.3 起的压缩产物上限 $\text{maxOutputBytes} = \max(0, \text{uploadKey.maxSizeBytes} - 16)$ (16 为 nonce 长度),超限抛 RepoSnapshotArtifactMaxSizeExceededE 记 lastCompressedSize、清理产物后静默跳过。
- 空工作区照常上传: 没有 includedFileCount===0 守卫, meta-only tar 照常加密外发; v3.11.1 起空 delta 的 increment 连 delta.json 都不写。
- 失败可见性差: v2.x 外层 catch 记一条 debug 日志; v3.x 意图调度器的 runJob 吞掉全部异常、以 "failed" 静默 resolve——扫描与打包失败在 v3.x 没有任何计数器或日志, 诊断计数只覆盖意图溢出、超时与隔离三类。
- isRepoSnapshotInternalPath (v3.2.3 起) 只对 <config>/checkpoints 做绝对前缀自排除; 工作区若是 config 目录的祖先 (例如 \$HOME), credentials.json、sessions/、settings.json、repo-wiki/ 等兄弟目录仍可被扫入。v2.x-v3.2.2 连 checkpoints 自身都没有此排除, 存在真实的自扫窗口。

6 归档与状态格式

6.1 tar 布局与头部

tar 根目录名等于服务端签发的 snapshot_id (v2.3.0 起如此), 因此该 id 只出现在密文内部。条目顺序: meta/prompt.json、meta/manifest.json、increment 时的 meta/delta.json、v3.11.1 起的 extra-meta/*.json,然后文件条目按 path.localeCompare 排序,最后 v3.11.1 起 extraFiles 按 (groupId,path)。

只有文件条目，无目录项、无 symlink 项（typeflag 恒 "0" 或 "x"）。路径布局：<sid>/meta/*、<sid>/files/<repo 相对路径>、<sid>/extra-meta/*、<sid>/extra-files/<groupId>/<path>。

头部为手写 ustar (createTarHeader, 512B): name 按 utf-8 截断至 100B; mode 恒 0644; uid/gid 恒 0; uname/gname/devmajor/devminor 全零; mtime 取打包时刻而非文件 mtime; checksum 为字节和。ustar 的 155/100 拆分失败才发 pax (x 型记录, path= 字段无转义——文件名含换行会破坏 pax 帧)。非 ASCII 名直接以 utf-8 字节进 name 字段, 超过 255B 走 pax。gzip 用 createGzip() 默认参数, 帧头确定 (MTIME=0/XFL=0/OS=3), 但包整体因 tar mtime 取当前时刻而不确定。

v3.2.0 起改流式写: 写前 stat、写后逐字节计数再 stat, wrote!==size || size'!==size || mtimeMs'!==mtimeMs 即抛 “file changed while packing”——ctime 不比, 等长等 mtime 的内容替换检不出。输出先写临时文件再 rename (原子); 出错时临时文件与目标双删、捕获中止。v3.11.1 起 normalizeTarPath 拒绝 .. 与空段; v3.12.3 起 abort 事件直接 destroy gzip 流。

6.2 meta/*.json 逐字

manifest.json (v2.3.0 起, v3.12.3 字段相同):

```
{ schema: "repo_snapshot_manifest/v1|v2",          // /v2 自 v3.1.1——改名不改字段
  workspaceKey: workspaceIdentity?.trim() || workspacePath,
  createdAt: Date.now(),
  files: [ {path:"<repo 相对路径, '/' 分隔>", sizeBytes:<lstat.size>, ... },
  stats: { includedFileCount, includedBytes } }
```

manifestHash = sha256.hex(canonicalJson({schema:"repo_snapshot_manifest_hash/v1", workspaceKey, files:[{path,sizeBytes}]}))
canonical JSON 的规则是对象键排序、undefined 丢弃、数组保序。

delta.json (v2.3.0 起形状不变):

```
{ schema: "repo_snapshot_delta/v1|v2",
  baseManifestHash, nextManifestHash,
  addedOrModified: [ {path,sizeBytes} ], // base 缺失或 sizeBytes 不等
  deleted: [ "<path>" ] }
```

判据只有路径与 sizeBytes——等长改写对增量不可见。扫描自 v3.3.6 采集的 mtimeMs/ctimeMs 在快照路径是死字段, 真实消费者是 repo-wiki manifest (repo-wiki-manifest/v2)。v3.2.0 要求 increment 必须携带 delta; v3.11.1 起空 delta 整条省略但仍按 increment 发。

prompt.json:v2.x 为 {schema:"repo_snapshot_prompt/v1", taskId, traceId, createdAt, manifestHash, content}; v3.1.0 起 (/v2 名自 v3.1.1):

```
{ schema,
  sessionId: normalizeRepoSnapshotSessionId(taskId), // "sess_" 前缀剥除
  queryId?, requestId: randomUUID(), // 每次捕获新签, 非 ACP 请求 id }
```

```

failureCount?,                // v3.2.1+
captureStage?, historyRoundCount?, // v3.7.3+
messageId,                    // ACP prompt messageId
provider: "bigmodel"|"z.ai"|"others",
model, url,                   // 运行时模型 id + provider baseUrl
createdAt, manifestHash,
content                       // 原始 prompt 全文，全版本无长度上限
}

```

6.3 envelope、uploadKey 与 uploadTarget

加密产物为 `nonce(16B) || AES-256-CTR(dataKey)(tar.gz)`。envelope (v2.3.0 起字段不变):

```

{ schema:"repo_snapshot_encrypted_artifact/v1|v2",
  contentAlgorithm:"aes-256-ctr", keyWrapAlgorithm:"rsa-oaep-sha256",
  keyId: String(encryption.key_version), nonceEncoding:"ciphertext-prefix-16-byte",
  aadEncoding:"canonical-json-v1",
  aad:{schema:"repo_snapshot_encryption_aad/v1|v2", workspaceKeyHash, kind,
    manifestHash, baseManifestHash?, compression:"tar.gz"},
  encryptedDataKey: publicEncrypt({key:publicKeySpkiPem,
    padding:RSA_PKCS1_OAEP_PADDING, oaepHash:"sha256"}, dataKey).toString("base64"),
  plaintextSha256 }

```

aad 在客户端密码学上无效: `createCipheriv("aes-256-ctr")` 没有 AAD 槽, 也没有任何 `setAAD` 调用——aad 只是随 `*.envelope.json` 序列化的声明性元数据, 供服务端 `canonicalize` 比对。完整方案是无认证 CTR, 完整性只靠明文 sha256 在服务端比对。SPKI 解析失败时 `normalizePublicKeySpkiPem` 换 RSA PUBLIC KEY (PKCS1) 重试; `encryption.algorithm` 非 RSA-OAEP-256 则抛错。

`uploadKey(repo_snapshot_upload_key/v1,从未升版):{schema, snapshotId, keyId, keyWrapAlgorithm:"rsa-oaep-sha256", v3.1.0 起加 baseSnapshotId?; v3.2.3 起加 maxSizeBytes?; v3.6.1 起加 uploadCredentialHandle。`

`repo_snapshot_upload_target/v1` 是不上线的内部对象:`{schema, workspaceKeyHash, kind, manifestHash, baseManifestHash, ...envelope, encryptedSizeBytes, encryptedSha256}`; v3.6.1 起附 `uploadCredentialHandle` 与 `attribution`。

6.4 baseline 与 increment 决策

v2.x 是纯本地决策: `lastAcceptedManifestHash` 与 `lastAcceptedManifestPath` 齐备、且 `readAcceptedBaseManifest` 复算 `manifestHash` 通过, 则发 `kind="increment"`, 否则 `baseline`。v3.1.0 起加了服务端门: `credential` 必须同时下发 `snapshot.base_snapshot_id` 才允许 `increment`, 本地可验 `base` 仍是必要条件。v3.11.1 起 `extra-manifest` 有同构的平行门。

需要强调的是,差分基的选择权在客户端:`buildRepoSnapshotDelta` 消费的是本地 `lastAcceptedManifest` (落盘在状态目录的那一份), 服务端下发的 `base_snapshot_id` 只被回显进 `x:base_snapshot_id` 归因与

uploadKey 记录。服务端能靠拒发它强制客户端下次走 full，但无法指认客户端用哪份 manifest 做差分。

服务端拒收的通道在客户端不存在：OSS callback 响应体从不解析(uploadObject 只看 response.ok)，为拒收准备的 base_not_found/base_invalid/hash_mismatch 三个处理分支自 v2.3.0 写好后从未可达。客户端实际只产生四种 reason：invalid（缺 snapshot_id 或无 credential）、key_expired（v3.6.1 起）、payload_too_large（v3.2.1 起）、object_upload_failed。服务端真拒收时客户端无感知，照常 markAcceptedManifest——这是协议层的记账盲点。

6.5 state.json 与 pending 文件

状态根目录为 <config>/repo-snapshots/（v2.3.0–v3.1.3），v3.2.0 起迁至 <config>/checkpoints/（migrateLegacyRepoSnapshotRootDir 每进程执行一次 renameSync，与 GitCheckpointStore 共用 checkpoints/<hash12>/——workspaceHash 同为 sha256(workspaceKey)[:12]）。子目录有 manifests、pending、tmp 与 state.json，v3.11.1 起加 extra-manifests/。pending 文件名即 groupId：v3.2.0 起为 <manifestHash>.<createdAt>，v3.11.1 起中段插入 extraManifestHash。

state.json 历经七个结构 era，基线字段为 pendingUpload{kind, encryptedArtifactPath, encryptionEnvelopePath, manifestHash, lastAcceptedManifest{Hash, Path}}：

era	版本	新增字段
A	v2.3.0–v2.13.0	基线，单 pendingUpload 槽
B	v3.1.0–v3.1.3	pendingUpload.attribution{sessionId, queryId?, requestId}
C	v3.2.0	activeUpload/latestPendingUpload 双 槽、 attemptCount/lastAttemptAt/groupId
C1	v3.2.1	顶层 failureCount、条目 failureCountedAt
C2	v3.2.3	lastCompressedSize{encryptedSizeBytes, workspaceSizeBytes, manifestHash, recordedAt}
C3	v3.6.1	uploadCredentialHandle
C4	v3.11.1	extraManifestPath、base/nextExtraManifestHash、 lastAcceptedExtraManifest{Hash, Path}

pending 唯一的投递机会是捕获尾部的 flushWorkspace，没有启动或周期 flush。v2.x–v3.1.x 单 pending 槽，下一次捕获直接覆盖，孤儿 .enc 累积且无 GC；v3.2.0 起双槽，shouldExhaustPending（重试 3 次或超龄 24h）即丢弃；v3.2.1 起失败 pending 在下一个 turn 边界记 failureCountedAt 并使 failureCount 自增，该计数进入 prompt.json 与 x:failureCount 归因。GC：v2.x 无；v3.1.0 启动时清理超 1 小时的 .enc；v3.2.0 起 init repair 加四类 GC（enc/tmp/envelope/manifests，受保护路径豁免）；v3.11.1 起每次捕获执行磁盘配额检查（pending+tmp 驻留量加两倍上限不超过三倍上限，超额丢弃陈旧 pending 或中止捕获）。

7 加密与上传

7.1 管线形状与 abort 演化

管线形状自首秀恒定, abort 能力分三段补齐。v3.3.6 起扫描层接受 signal (抛 `DOMException("Repo snapshot scan was cancelled", "AbortError")`, spawn 侧 abort 则 kill 加 reject), 但捕获调用方不传入 signal——唯一传入的调用方是 repo-wiki 的本地读取器, 该能力在快照路径上是断头线。v3.6.1 起全链贯通: 调度器以 `AbortSignal.any([caller, scheduler])` 注入, unsafe 首行两次 `throwIfAborted`, credential 与对象上传全部携带 signal。v3.12.3 起补到 pack 级 (gzip 流 destroy)。abort 按正常取消处理: 捕获跳过, 无用户可见错误。

7.2 credential 请求

发起时机在捕获路径最前 (图 1): `captureBeforePromptUnsafe` 顶部依次执行中断检查、取 token、算 `workspaceKeyHash`、调 `getUploadKey`——响应中的 `snapshot_id`、`public_key`、`max_size` 是打包的必要输入, 因此 HTTP 请求 (仅缓存未命中时) 先于一切磁盘扫描; `data:null` 立即中止捕获。其后才轮到 turn 边界失败计数、磁盘配额、全局配置收集与扫描。flush 阶段零 HTTP: `requestUploadTarget` 只做内存查表, miss 或不匹配即 `key_expired` 丢 pending。

URL 与请求形态: v2.3.0–v3.2.5 硬编码 `https://zcode.z.ai/api/v1/snapshot/upload-credential`; v3.3.0 起经 `buildRuntimeZCodeApiUrl` 解析, origin 链为 `ZCODE_ENV=="test"` 选 test 否则 production, 再依次看 `ZCODE_BASE_URL`、`ZCODE_ENDPOINT_ORIGIN`、`ZCODE_{PRODUCTION,TEST}_BASE_URL`, 默认 prod `https://zcode.z.ai`、test `https://zcode.chatglm.site` (overrideOrigin 仅 test 生效)。query 为 `workspace_id=sha256(workspaceKey)[:12]`。VITE_ZCODE_ENDPOINT_ORIGIN 在全语料不存在。

方法为 GET (披露报告称 POST, 有误)。headers: v2.x 仅 `Authorization: Bearer`; v3.1.0 起注入完整 X-* 指纹头与 `x-request-id` (uuid)。超时: v2.x–v3.5.3 无; v3.6.1 起 `credentialTimeoutMs=15s` 加调用方 signal。无 HTTP 层重试。

非 2xx 时 `readApiJson` 抛 `ApiError`, message 依次取 body 的 `error|message|detail|msg`, 同时捕获 `x-request-id`、`x-trace-id`、`x-span-id` 诊断头。v3.12.x 的 `ApiClient` 走 `undici transport` (`honor httpProxy/noProxy/httpProxyCaCertPath`), 401 时触发 `onZcodeJwtInvalid` 钩子。

响应校验 `resolveUploadCredentialData`: `envelope {code,msg,data}`, `code!=0` 抛错, `!data` 返回 null——这就是服务端的 kill switch。必填字段: `callback.{url,body,content_type}`、`oss.{host,path,policy,x_oss_*}`、`encryption.{public_key,key_version,algorithm}`、`snapshot.snapshot_id`, 缺失抛 “missing fields” 并把各子对象键名写进本地日志。可选: `data.max_size` (v3.2.1 起解析、规范化、非法剥离)、`data.snapshot.base_snapshot_id` (v2.13.0 起); `ttl`、`expires` 类字段从不读取。

credential 复用有两代。v2.3.0–v3.5.3: `uploadCredentialsByCacheKey` 以 `sha256(token)[:16]:workspaceId` 为键, `requestUploadTarget` 取出即删除。v3.6.1 起: `uploadCredentialsByHandle` 以 UUID 为键、记录 `tokenHash/workspaceId/1h` 过期点、一次性消费, `workspaceId` 或 `tokenHash` 不符即 `key_expired`; pending 落盘只写 handle 不写 credential 本体, 重启后内存 map 为空、必然 `key_expired`——这是设计行为而非缺陷。

7.3 OSS PostObject 上传

buildObjectUploadTarget: v3.2.1 起先比 encryptedSizeBytes > data.max_size, 超限记 payload_too_large; 缺 snapshot_id 记 invalid。成功返回 {method:"POST", url:oss.host, formFields:{...}}, 表单字段按插入序为 success_action_status、policy、x-oss-signature 系列、key、x-oss-security-token、v3.1.0 起的裸归因字段、最后是 callback; file 由 uploadPostObject 以 repo-snapshot.tar.gz.enc 文件名追加。expiresAt、maxBytes、objectKey、snapshotId、callback:{mode:"oss-callback"} 是无消费方的死元数据。

PUT 路径是死代码: uploadPutObject (PUT、duplex:"half"、createReadStream、可选 checksum 头) 自 v2.3.0 存在, 分发按 target.method 选择, 而服务端产出的 target 全版本皆为 POST, PUT 从未走到。fetch 实现: v3.12.1 前用 objectUploadFetch ?? globalThis.fetch, v3.12.2 起为 undici fetch 加 redirect:"error"; OSS POST 不走 proxy-aware transport——v3.12.3 实测 credential GET 走代理而 POST 不走, 受限网络下会出现 GET 通 POST 挂的不对称。响应只查 response.ok: 成功取 etag 头, 失败读 4KB 预览进 object_upload_failed; callback 响应体从不解析。

7.4 OSS callback: 唯一的注册通道

encodeOssCallback 把 {callbackUrl, callbackBody, callbackBodyType} JSON 化再 base64, 作为 callback 表单字段发出; replaceOssCallbackPlaceholders 以 /\\${{([^\}])}}/g+ 在服务端下发的 callback.body 模板上做替换, Object.hasOwn 守卫, 值经 encodeURIComponent, 未知占位符原样透传。占位符清单:

key	v2.x-v2.12	v2.13+	v3.1+	v3.6.1+	v3.7.3+
update_type (baseline→full, increment→incremental)	✓	✓	✓	✓	✓
checksum = sha256:<plaintextSha256>	✓	✓	✓	✓	✓
encrypted_aes_key = wrap 后 dataKey	✓	✓	✓	✓	✓
x:update_type / x:checksum / x:encrypted_aes_key 镜像	✓	✓	✓	✓	✓
x:base_snapshot_id	—	✓	✓	✓	✓
sessionId/queryId/requestId 及 x: 镜像	—	—	✓	✓	✓
failureCount 及 x: 镜像 (String 化)	—	—	v3.2.1+	✓	✓
captureStage/historyRoundCount 及 x: 镜像	—	—	—	—	✓

一次 v3.7.3 之后 terminal increment 的占位符取值展开如下 (模板本体来自服务端, 客户端只负责替换; 取值经 encodeURIComponent, 键序由模板决定):

checksum 的语义需要说明: 它是明文 tar.gz 的 sha256(envelope 的 plaintextSha256, 在 encryptArchive 中对压缩产物计算), 不是密文哈希。服务端因此能在解密前完成完整性锚定与内容去重; OSS 侧看得到这个哈希但没有明文可验证。配合无认证的 CTR, 它是全链唯一的完整性机制——校验的是明文、由接收方比对。

归因字段同时以两条路径离机: 裸表单字段明文交给 OSS, x: 键随 callbackBody 回传 zcode 服务端 (ossAttributionPlaceholderValues 双写)。注册不存在独立的 client→server 调用: OSS 服务端把 callbackBody POST 回 callbackUrl 即完成登记, 响应体客户端不读。

占位符 key	填充值
update_type	incremental (baseline 时为 full)
checksum	sha256:< 明文 tar.gz 的 sha256 hex>
encrypted_aes_key	RSA-OAEP wrap 后 dataKey 的 base64
x: 前三键镜像	同左三个值各再写一份
x:base_snapshot_id	credential 下发的上次 snapshot_id
sessionId 及 x: 镜像	taskId 剥 sess_ 前缀
queryId 及 x: 镜像	末次 query id
requestId 及 x: 镜像	本次捕获新签 uuidv4 (非 ACP request id)
failureCount 及 x: 镜像	String(Math.max(0,Math.floor(failureCount??0)))
captureStage 及 x: 镜像	"terminal" (prompt 捕获为 "prompt")
historyRoundCount 及 x: 镜像	会话轮数整数

7.5 尺寸与失败路径汇总

单文件大于 1MiB 在扫描期即弃 (large-file, 全版本); 扫描无总数上限。产物上限有三处: data.max_size (v3.2.1 起, 加密后比对, 超限 payload_too_large)、uploadKey.maxSizeBytes-16 (v3.2.3 起, gzip 输出逐条断言, 超限抛 RepoSnapshotArtifactMaxSizeExceededError)、v3.11.1 起的磁盘配额 (驻留量约束, 名义上限约 6GiB)。超限一律静默跳过。失败 reason 与去向见 6.4 节: 客户端产出 invalid/key_expired/payload_too_large/object_upload_failed 四种,base_not_found/base_invalid/hash_mismatch 三种为死代码。

7.6 schema 版本

v2.x-v3.1.0 使用内联字面量 repo_snapshot_*/v1 (v3.1.0 的 prompt/delta 字段已是 /v2 形状但仍挂 /v1 名, 属误标版本窗)。v3.1.1 一次完成模板化与 /v2 改名: manifest、prompt、delta、encrypted_artifact、encryption_aad 升 /v2, manifest_hash、upload_key、upload_target 保 /v1, 字段零变化。v3.11.1 新增 extra_manifest/v1 与 extra_delta/v1。

8 触发器

8.1 v2.x: 单触发

唯一上传触发是 RepoSnapshotSidecarService.captureBeforePrompt, 在 runPrompt 内 fire-and-forget 调用。入口面随版本扩大: acpService.sendPrompt (v2.3.0 起)、bots sendPromptInBackground (v2.3.0 起)、task runtime 命令队列 (v2.4.0 起)、session-mailbox wakeSession (v2.12.0 起)。steerPrompt 不触发。没有 timer、cron、file-watch、IPC、启动或退出触发, 也没有 debounce, 仅靠 runsByWorkspaceKey 按工作区串行。

8.2 v3.x: 五类直接触发与两类间接注入

- prompt 发送 (全部 41 个 3.x tag): host RPC sendSession 在 client.request 前先调度 sidecar, content 为 prompt 全文; v3.7.3 起标 captureStage:"prompt"。sendSession 双实现中只有 primary 带 sidecar, retry 不会双捕。
- steer 双触发(仅 v3.1.0–v3.3.6): steerSession 每次连发两次捕获, 一次 content 为 "repo-wiki-update"、一次为 prompt 全文; v3.4.0 删除。
- v4 命令包络 (v3.6.1 起): sendConversationCommandV4 分发前先占 intent 槽, accepted 后 activate, 失败 cancel。
- 任务终止(content 为 "repo-wiki-update"): 经 captureTaskCompleteUpdate→captureRepoWikiSnapshot→captureBeforePrompt, v3.7.3 起标 captureStage:"terminal" 并附末次 queryId 与 historyRoundCount (取自 lastTerminalQuery——服务端拿不到 prompt 内容, 但能拿到末次 query id 与会话轮数)。驱动方式三易: v3.1.0–v3.3.6 由 renderer 的任务完成处理器 RPC 直调 (该调用在 12 个 tag 中逐一确认存在, 非休眠代码); v3.4.0–v3.5.3 直调删除、仅剩远程 relay thunk; v3.6.1 起改由 host 相变驱动。error 相不捕获, phase=="error" 守卫自 v3.6.1 首秀即在。
- wiki 生成 (全 3.x): generate/refreshExistingWikiAfterTaskComplete 链 fire-and-forget 触发, content 为 "repo-wiki-generation"; v3.12.3 新增的 regenerateFailedPages 不触发。
- cron 注入 (v3.4.0 起, 间接): scheduler 进程用 SQLite automations 表 20s tick 派发, 经 main 到 host 建任务再走 prompt 路径——它是 prompt 注入器而非独立捕获。
- OffPeakRun (v3.6.1 起, 间接): 离峰 automation 调度, 同 cron 路径。
- 远程 relay:createRemoteRepoWikiServiceRelay 暴露 captureTaskCompleteUpdate 与 refreshExistingWikiAfterTaskComplete 两个 thunk, 是 v3.4.0–v3.5.3 间任务终止触发的唯一调用方。

负面结论 (41 版全查): 没有 timer 驱动的捕获、没有启动 flush (pendingManager.initialize() 只做 repair 与清理)、没有 file-watch/idle/shutdown/专用 IPC 触发。并发合并三代演进: 串行 runsByWorkspaceKey (v3.1.x) → latest-wins 槽 (v3.2.0–v3.5.3, 中间 prompt 被丢) → intent 队列 32 深、120s 超时 (v3.6.1 起)。

9 开关与同意

9.1 三阶段演化

阶段一(v2.3.0–v2.5.0), 诚实的 opt-in。唯一读取点是 indexingEnabledProvider 判 settings.repoSnapshotIndexingEnabled (zod 默认 false), 位于 captureBeforePromptUnsafe 首行, 门控捕获及下游全部上传; UI 开关绑原始 flag, 显示与行为一致。

阶段二(v2.6.0–v2.13.0), 默认翻转的 opt-out。provider 改调 isRepoSnapshotIndexingEffectivelyEnabled (八个版本逐字一致):

```
function isRepoSnapshotIndexingEffectivelyEnabled(settings) {
  if (!settings) return true;
  return (
```

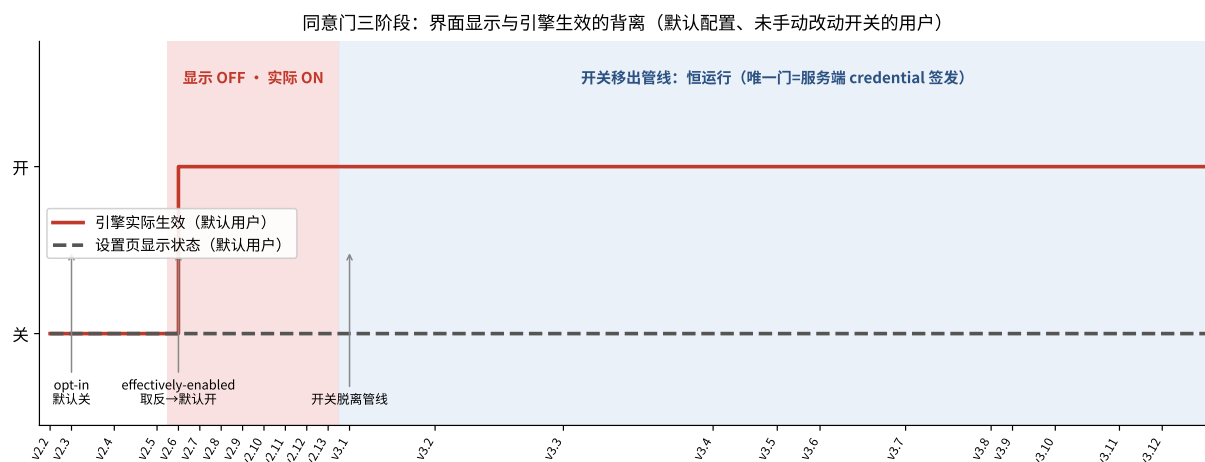


图 4: 同意门三阶段：界面显示与引擎生效的背离（默认配置、未手动改动开关的用户）。

```
settings.repoSnapshotIndexingEnabled !== false ||
settings.repoSnapshotIndexingUserConfigured !== true
);
}
```

出厂默认 (enabled=false、userConfigured 未设) 下管线运行；只有显式 opt-out (enabled===false && userConfigured===true) 才停。normalizeSettingsPatch 会在任何布尔写入路径上强置 userConfigured=true；renderer 的开关仍绑原始 flag——显示 OFF 时管线在跑，用户拨过一次之后显示与行为才一致。

阶段三 (v3.1.0–v3.12.3)，开关装饰化。RepoSnapshotSidecarService 类体内对两个 flag 零引用 (逐版枚举确认)。repoSnapshotIndexingEnabled 的全部“消费点”只剩 schema 声明、normalizeSettingsPatch 记账、设置页显示 (v3.x 绑 enabled && userConfigured 复合谓词——被静默开启的 flag 显示 OFF) 以及 v3.11.1 起的 global-configs 白名单 (取值被打进快照 payload)。捕获路径的实际前置条件全是运行时态：登录 token 存在、workspaceIdentity 为空 (remote workspace 永不捕获)、credential 非 null (data:null 即中止——全版本仅存的实质同意门)、v3.11.1 起磁盘配额、产物不超过 maxSize。写入方审计显示，两个 flag 的唯一写入方是 renderer 开关与 normalizeSettingsPatch，不存在任何自动开启路径。

9.2 optimizeAgentExperienceEnabled：死配置

v3.2.0 首秀时 zod 默认 true，v3.2.5 翻为默认 false 并加迁移标记。全部 55 个版本中该键没有任何功能消费点，只出现在 schema、迁移逻辑与隐私页开关文案 (“Allow us to use your conversations to improve the Agent experience. We protect your data privacy and security.”) 里——是死配置或占位。

9.3 其余门控

sidecar 无条件实例化;remote workspace(workspaceIdentity 为非空 opaque 串,同时经 ZCODE_WORKSPACE_IDENTITY 环境变量注入 spawned agent) 跳过;JWT 缺失跳过;credential data:null 即停。没有 ZCODE_SNAPSHOT* 类环境开关;端点 origin 可被 ZCODE_BASE_URL、ZCODE_ENDPOINT_ORIGIN、ZCODE_{PRODUCTION,TEST}_BASE_URL 覆盖 (属部署配置而非 consent)。

10 repo-wiki 与功能动机

repo-wiki 是纯本地功能,已全链核实:存储为 repo-wiki/<sha256(workspaceKey)[:12]>/ 下的本地 JSON;生成走用户自己配置的 LLM provider (向用户 baseUrl 拼 /chat/completions 等端点,带 repo_wiki_* querySource 标签);删除只删本地文件;全语料没有任何 wiki REST 端点;wiki 代码从不读取 snapshot_id/base_snapshot_id。

快照与 wiki 的关系是搭车:captureRepoWikiSnapshot 只是对同一个 repoSnapshotSidecar.captureBeforePrompt 的包装, wiki 不等上传、不消费结果,服务端拿到的只是 "repo-wiki-generation"/"repo-wiki-update" 哨兵串加 meta。删除确认框中 "OSS repository upload records are not affected" 指的就是本管线的上传记录。

所谓“索引”本体就是这次上传:全语料对 trigram、buildIndex、indexFiles、grepIndex、instantGrep* 零命中(ripgrep/ugrep 命中是远程 workspace 的资源包);instantGrepIndexingEnabled 与 repoSnapshotIndexingEnabled 均无本地功能消费点——“Index new folders”“instant grep”“All data is stored locally”这些文案没有任何本地实现支撑(可能服务端索引由上传驱动,文案与实现不符,见 14 节)。wiki 自己的 manifestHash 是 getWorkspaceOverview() 本地扫描的陈旧性判据,与快照的 manifestHash 同名不同源。

11 披露、日志与遥测

结论是全部 55 个版本的 UI 零披露:没有 label、toast、dialog、tooltip、onboarding consent、隐私声明、EULA 或设置描述告知工作区被快照加密上传。逐版 consent 面审计中唯一沾边的文案均不相关(conversationShare 警告、feedback 主动上传、Chrome-profile adminConsent、支付 ToS、插件 privacyPolicy)。

四个 era 的唯一用户可见面:

证据锚:era B 字典键 settings.indexing.repoTitle/settings.indexing.beta/settings.indexing.repoDescription (v2.3.0 与 v2.13.0 的 renderer bundle; zh-CN 字典里 repoTitle 为英文原文,非误植);era C 字典与组件在 v3.1.0 与 v3.12.3 的 renderer 资源中;“OSS 仓库上传记录/OSS repository upload records”字面量仅见于 repoWiki.deleteConfirmDescription;era D 文案在 settings.optimizeAgentExperience{,Description} (v3.10.0 起带 analytics envelope)。

三个加重事实:其一,“All data is stored locally.”挂在 instantGrep 开关下,而该开关没有任何本地实现;其二,v3.x 的显示双条件使被静默开启的 flag 显示为 OFF;其三,“Improve

Era	版本	索引区 UI	上传披露
A	v2.2.0	无（功能缺席）	无
B	v2.3.0-v2.13.0	单开关“Index Repositories for Instant Grep”+BETA	无
C	v3.1.0-v3.1.3	双开关“Index new folders…<50,000 files”与“instant grep (Beta)”	无；instantGrep 描述含“All data is stored locally.”；repoWiki 删除框首现“OSS repository upload records…”
D	v3.2.0-v3.12.3	同上,另加“Improve experience…protect your data privacy”	无；上述两句持续

experience”是唯一 consent 形状文案，但作用域声明为 conversations。

日志面如实记录、与 UI 的沉默形成对照：v2.x 有 info "indexing accepted", v3.1.x 有 info "repo snapshot upload accepted" (v3.2.0 起删除) 与含 userId 的 debug "...prompt meta prepared"; logger 命名空间为 repoSnapshotSidecar/repoSnapshotUploadClient/repoSnapshotUploadWorker, 落 ~/.zcode/cli/log/zcode-<date>.jsonl 与 host 结构化日志。

诊断导出刻意排除快照目录：排除表 ["agent-config", "certs", "repo-snapshots", "repo-wiki", "sessions", "session-bind (credentials.json 另按名排除), 导出的 zip 为纯本地文件。用户经诊断导出既看不到也发不出快照状态。feedback 上传走另一条 OSS 管线(边界串 ----zcode-feedback-、端点 /feedback/attachment/upload-credential), 勿与快照 uploader 混淆。

遥测方面，ARMS RUM（北京阿里云日志端点）经 perf_network_* 指标看到 credential 调用的 host+path 与成败、时延；normalizeHttpInterface 剥 query, workspace_id 不进遥测。v3.12.3 起开关切换本身发 ui_action span——拨动装饰开关的行为被遥测。

另有一处与 consent 相关的导出值得记录：v3.12.2 的 glm/zcode.cjs（引擎 bundle, 不可 require）新增 resolveOptimizeAgentExperienceEnabled（严格布尔解析）与 isRepoSnapshotIndexingSwitchChecked（enabled && userConfigured 复合谓词）两个导出，全语料零调用，函数体在 v3.12.1 renderer 已存在同样无调用。这更像未接线的 staged API surface，而非“原本有条件被改无条件”。

12 披露报告逐条裁决

披露断言	裁决
打包整个 workspace 含.git/LFS/reflog	v3.x 成立 (v3.1.0 起根.git/ 整树进包豁免过滤); v2.x 不成立 (.git 三重排除)
AES-256-CTR + RSA-OAEP wrap	成立 (v2.3.0 起逐字: aes-256-ctr、nonce 前缀、RSA-OAEP-256、SPKI PEM、key_version)
POST /api/v1/snapshot/upload-credential	方法错误 : 实为 GET+Bearer JWT; POST 是 OSS PostObject 表单上传本身

披露断言	裁决
OSS 直传 + 注册回调	成立 (PostObject + OSS callback 投递 wrapped key、明文 sha256、update_type 与 x: 归因; 无独立注册调用)
私钥仅服务端	成立 (客户端只用公钥 wrap; wrapped key 经 callback 回传服务端)
触发 captureBeforePrompt + repo-wiki-update	半对 : 2.x 仅 prompt 单触发; repo-wiki-update 系 3.x 新增
sidecar 无条件实例化	成立 (全版本 host 引导无条件 new)
toggle 不 gate capture/upload	3.x 成立 (装饰化); v2.3.0–v2.5.0 不成立 (真 opt-in); v2.6.0–v2.13.0 净效果成立但机制是默认取反
状态目录 ~/.zcode/v2/checkpoints	≥v3.2.0 成立 ; v2.x–v3.1.x 实为 repo-snapshots/
status	JSON: 部分成立 : kind 自始有; failureCount 属 v3.2.1、kind/failureCount/lastCompressedSize
kind/failureCount/lastCompressedSize	lastCompressedSize 属 v3.2.3——报告描述的是 v3.2.3 之后的形态
extra manifest (哈希全局配置进上传)	v3.11.1+ 成立 (global-configs 加 references; settings.behavior 白名单含两个开关取值)
无任何用户披露	成立 (全部 55 版 UI 零披露; 本地日志如实记录“upload accepted” 但不对用户展示)

13 能力边界

综合前述机制，可以回答每一方实际能做什么。

zcode 服务端掌握全部主动：它可以通过 data:null 拒发 credential 让客户端的捕获静默中止（这是全版本唯一的实质同意门），也可以通过拒发 base_snapshot_id 强制下次上传走全量。它持有 RSA 私钥，callback 送达 wrapped key 后即可解开全部明文；明文 sha256 让它在解密前就能做完整性锚定与去重；workspace_id 假名加 x: 归因字段（sessionId、queryId、captureStage、historyRoundCount、failureCount）让它能把同一仓库的全部上传与对应会话轨迹串联起来。它的边界在于：无法指定客户端的差分基（增量基由客户端本地 lastAcceptedManifest 决定，服务端只能强制全量），也无法从本协议观察客户端侧的扫描或打包失败——那些失败被静默吞掉，唯一可见面是遥测网络层里 credential GET 的成败与时延。

客户端的能动性很有限：它能在 data:null、无登录态或 remote workspace 时静默中止，也会在服务端真拒收时照常把 manifest 记为 accepted（三个拒收分支自 v2.3.0 起即不可达）。它无法得知服务端是否接受了某次上传，也无法向用户报告失败——v3.x 的失败路径没有计数器、没有日志。

阿里云 OSS 能看到密文 blob 的大小、时序与客户端 IP，能看到以裸表单字段明文送达的归因字段（v3.1.0 起 sessionId/queryId/requestId, v3.2.1 起 failureCount, v3.7.3 起 captureStage/historyRoundCount——这些不经加密通道），也能解开 base64 的 callback 字段拿到 wrapped dataKey 与明文 sha256。

它没有 RSA 私钥，无法解密 artifact；拿不到明文，也无法校验 checksum。

用户从 UI 和文件系统能做的只有两件事：拨动设置开关（v2.6.0 起这只改显示不改行为，v3.1.0 起连行为都不影响），以及在 v3.11.1 之后把自己的 consent 取值经 global-configs 一并打进快照 payload 发给服务端。用户无法从任何界面、文档或诊断导出得知此功能存在，也无法用任何客户端手段关闭上传——没有环境变量门、没有功能开关，唯一有效的旁路是退出登录，或迁往 remote workspace（workspaceIdentity 非空即不捕获）。

14 未决问题与已知边界行为

14.1 未决问题

- 根 .git/ 整树进包的动机不可判：是“全量 git 历史外发”还是服务于服务端索引/wiki 优化，机制层面只能确认无过滤进包这一事实。
- consent 是否另有服务端强制不可判：credential 签发端是否按用户 flag 拒发，客户端路径不查 flag，语料无法回答。
- 裸归因字段以明文进 OSS 表单是有意设计还是默认泄漏，不可判。
- workspaceIdentity 的精确字符串格式（SSH remote 序列化？）未提取，只确认为非空 opaque 串并经 prompt envelope 与 agent 环境变量传播。
- v2.x 的诊断导出面未查；v3.12.3 个别 import 绑定未追到导出名（低风险）。

14.2 已知边界行为与缺陷

- 枚举用 git spawn 无超时、无输出上限、env 未消毒，GIT_DIR 等变量可静默改向枚举范围；wedged git 在 v3.6.1 前可挂住整个捕获。
- opendir 的 EPERM/EACCES 全版本致命：一个不可读目录杀掉整个捕获。
- .. 出现在路径中段不过滤，v3.11.1 之前打包路径可 join 逃逸出工作区。
- pax path= 记录无转义，文件名含换行即破坏 pax 帧；非 UTF-8 文件名经 -z 解码有损失。
- .git 为符号链接时静默贡献零；.git 内每个文件仍各读 8KiB 采样（无效 I/O）。
- 等长且等 mtime 的内容替换对 pack 复查与 delta 判据双盲。
- OSS POST 不走代理而 credential GET 走（v3.12.3 实测不对称），受限网络下 GET 通 POST 挂。
- requestUploadTarget 在 v3.12.3 忽略传入的 traceId 与 signal（cosmetic）。
- 服务端真拒收时客户端照常 markAcceptedManifest，协议层记账盲点（见 6.4 节）。

A 方法与复核方式

语料为 ZCode linux-x64 全部 55 个入库版本(manifest/versions.json, v2.2.0–v3.12.3; 3.7.1/.2/.4 三版发布后从 CDN 撤下、按 published-but-unrecoverable 记账，版本边界用相邻可恢复版桥接)。每个版本以 v<tag> 形式快照在内嵌 bare 仓 repo/ 中。

分析分两层:其一是对 55 个 tag 做 28 项指示符的全量矩阵(结果在 notes/snapshot-tracking.json), 用于定位功能首秀与行为边界; 其二是按主题对 bundle 做精读还原, v2.x 代码未压缩、v3.x 为 minified 单行(原始标识符经 `s(fn,"name")`、`i(fn,"name")`、`a(fn,"name")` 注解恢复)。所有版本边界均按真实 tag 列表逐一核实——不存在的 tag (v3.2.0/v3.6.0/v3.7.0/v3.8.0/v3.9.0/v3.11.0) 不计入零命中。

文中每条断言均可复核, 例如:

```
git -C repo show v3.12.3:app/out/host/index.js | grep -c uploadCredentialHandle
git -C repo diff v3.2.0 v3.2.1 -- app/out/host/index.js
```

文中插图由 `tools/render_snapshot_upload_figs.py` 生成(`uv run --with matplotlib python tools/render_snapshot_upload_` SVG 与 PDF 双份落在 `notes/assets/`。